GUÍA DEL EQUIPO · 23 AL 25 DE OCTUBRE DE 2026

Pipeline Banco

Plan de trabajo, archivos, commits y dependencias

Para **Avilix, Maki y Leo**. Trabajo de viernes, sábado y domingo. Este documento explica qué construir, quién lo construye, qué entregar y cuándo se puede continuar.

Integrante y color	Rol obligatorio	Responsabilidad
MAKI · verde	Ingeniería 1: calidad	Diagnóstico, duplicados internos, faltantes, imputación e IQR/capping.
AVILIX · azul	Ingeniería 2: integración	Lectura, fechas, moneda, catálogo, Min-Max, segmentos y conexión del pipeline. También realiza los extras visuales.
LEO · morado	Ingeniería 3: validación y BD	Pydantic, cuarentena, SQLite, transacciones seguras y pruebas de resiliencia.

Regla del fin de semana: primero completar y probar el pipeline obligatorio. Después conectar dashboard, cajero visual y transferencia digital. El viernes solo Avilix crea y sube commits.

Cómo utilizar esta guía

• Todos leen las páginas 2 a 6: prioridades, preparación y contrato compartido.

• Maki sigue las páginas 7 y 8; Avilix, 9 y 10; Leo, 11 y 12.

• Las páginas 13 a 17 explican dependencias, Git, uso de IA y pruebas de integración.

• Las páginas 18 a 21 cubren extras, reporte, defensa y entrega final.

Estado de partida: el repositorio tiene REGLAS_DE_NEGOCIO.md v1.4. Los archivos de código enumerados aquí son entregables por crear; no se afirma que ya existan ni que las pruebas hayan pasado.

Repositorio: github.com/Av1I1X/Pipeline_Banco

Documento de coordinación v1.0 · Fecha de referencia: viernes 23 de octubre de 2026. El domingo es el cierre interno del equipo; la fecha oficial de entrega debe verificarse en la asignatura.

01 · TODOS

Qué es obligatorio y qué es extra

ETL significa extraer, transformar y cargar: leer archivos, corregir lo permitido, validar y guardar. **Cuarentena** es la tabla donde se conserva un registro rechazado y sus causas. Un registro limpio no equivale a una transferencia bancaria exitosa.

Prioridad	Entregable	Responsable principal
P0 · profesor	Perfilado inicial, nulos, duplicados, tipos y comparación antes/después.	Maki
P0 · profesor	Hipótesis MCAR/MAR, mediana justificada, IQR 1.5 y capping.	Maki
P0 · profesor	CSV/XLSX/JSON, fecha ISO, USD a MXN a 19.50, catálogo y huérfanos.	Avilix
P0 · profesor	Min-Max [0,1] y segmentos Bajo, Medio y Alto.	Avilix
P0 · profesor	BaseModel, @field_validator, ValidationError y motivo_rechazo_cuarentena.	Leo
P0 · profesor	SQLite, transacciones_limpias, cuarentena_errores, commit/rollback y estrés.	Leo
P0 · profesor	Roles en Git, diagrama, reglas firmadas, repositorio, BD, reporte PDF y defensa.	Los tres; Avilix coordina
P1 · alcance del equipo	Nuevos lotes, duplicados globales, catálogo versionado y reprocesamiento.	Leo + Avilix
P2 · extra visual	Dashboard, cajero ATM y transferencia tipo app; parámetros para ingreso manual.	Avilix; servicios de Leo

P0, P1 y P2 indican orden de construcción, no permiso para omitir funciones. P1 y P2 están en las reglas v1.4. Si el tiempo no alcanza, se documenta lo pendiente con honestidad; no se presenta una pantalla simulada como funcional.

La guía anuncia seis fases pero enumera siete: se implementan las siete. El ATM recibido es XLSX aunque el PDF menciona CSV; se documenta la diferencia y el lector admite ambos. No hacen falta cantidades ni precio unitario: esos campos son ejemplos de otro dominio.

02 · VIERNES · AVILIX

Preparar el espacio de trabajo

Objetivo de hoy: que Maki y Leo puedan empezar el sábado sin inventar nombres, formatos ni responsabilidades. Hoy ellos pueden leer y aceptar la invitación; no deben subir commits.

Archivo o acción	Contenido exacto / comprobación
README.md	Objetivo, nombres y roles, requisitos, instalación, comandos de ejecución y pruebas, alcance P0/P1/P2. Marcar los comandos como previstos hasta implementarlos.
.gitignore	Excluir .venv/, __pycache__/, .pytest_cache/, .env, secretos, tmp/, outputs/ y bases de trabajo. No excluir la entrega final delivery/pipeline_banco.sqlite.
requirements.txt	Dependencias: pandas, numpy, openpyxl, pydantic v2, email-validator, pytest y reportlab. Extras: FastAPI, uvicorn, python-multipart y httpx para pruebas. sqlite3 viene con Python. Fijar versiones comprobadas.
docs/CONTRATO_DATOS.md	Copiar y formalizar el contrato de las páginas 5 y 6: firmas, campos, tipos, errores, rutas y orden del proceso.
docs/PLAN_DE_TRABAJO.pdf	Copiar este PDF con ese nombre al repositorio para que todos consulten el mismo plan.
src/__init__.py; tests/__init__.py	Archivos vacíos que permiten organizar imports de código y pruebas.
datasets/README.md	Nombres de las tres fuentes, procedencia académica, ubicación e instrucciones para obtenerlas. Los originales no se editan.
Colaboradores	Invitar las cuentas reales de Maki y Leo desde Settings > Collaborators. Los apodos no bastan para adivinar sus usuarios de GitHub.

Fuentes previstas en datasets/: catalogo_cuentas.csv, transacciones_atm.xlsx y transferencias_digitales.json. Si está autorizada su publicación académica, Avilix las incorpora sin cambios; de lo contrario, las comparte por el canal del curso y deja en README cómo colocarlas localmente. El repositorio es público: no subir datos bancarios reales.

Commits del viernes, solo Avilix

```
docs: agregar plan de equipo y contrato de datos
chore: preparar estructura y dependencias del proyecto
docs: registrar fuentes y pasos de instalacion
```

El commit de reglas ya existe: no duplicarlo ni reescribir su historia. Antes de terminar el viernes, confirmar que Maki y Leo tienen acceso y pueden leer main.

03 · TODOS

Carpetas y propiedad de archivos

Todas las rutas son relativas a la raíz de Pipeline_Banco. Cada archivo tiene un dueño: los demás solicitan cambios mediante una revisión, en lugar de editarlo simultáneamente.

Dueño	Archivos principales	Para qué sirven
Avilix	src/lectura.py src/integracion.py src/transformacion.py src/pipeline.py src/cli.py	Leer fuentes; homologar fechas/divisas; cruzar catálogo; escalar y segmentar; coordinar etapas; ejecutar desde consola.
Maki	src/calidad.py src/duplicados.py src/limpieza.py tests/test_calidad.py tests/test_limpieza.py	Perfilar originales, separar duplicados del lote, imputar y calcular/aplicar IQR.
Leo	src/validacion.py src/cuarentena.py src/persistencia.py src/schema.sql src/catalogo.py src/consultas.py	Validar; construir rechazos; guardar; controlar cargas, claves globales y versiones; consultas para resultados.
Avilix	tests/test_integracion.py tests/test_transformacion.py tests/test_pipeline.py	Comprobar lectores, transformación y funcionamiento completo.
Leo	tests/test_validacion.py tests/test_persistencia.py tests/test_resiliencia.py	Probar Pydantic, rollback, idempotencia y datos corruptos.
Cada autor	docs/reporte_maki.md docs/reporte_avilix.md docs/reporte_leo.md	Resultados y justificaciones por especialidad, con números obtenidos del código.
Avilix	docs/diagrama_pipeline.mmd docs/diagrama_pipeline.png scripts/generar_reporte.py	Diagrama editable y visible; ensamblado reproducible del reporte final.

src/ contiene código; **tests/**, comprobaciones automáticas; **datasets/**, entradas; **outputs/**, resultados de trabajo ignorados por Git; **delivery/**, archivos finales seleccionados para entregar.

> No subir archivos sueltos llamados final2.py, prueba_nueva.py o copia_buena.py. No renombrar los módulos acordados. Si hace falta una dependencia o un archivo compartido, avisar a Avilix para actualizar el contrato.

04 · CONTRATO COMPARTIDO

Qué debe entregar cada función

Este es el contrato de implementación. Avilix lo coloca en docs/CONTRATO_DATOS.md antes de que los demás creen sus ramas. Los ejemplos de funciones son firmas acordadas, no código ya implementado.

Dueño / archivo	Firma y salida obligatoria
Avilix · lectura.py	leer_fuente(ruta, fuente) -> dict: registros (lista de dict), metadatos y errores_archivo. No limpia ni elimina filas. Una fila JSON no estructurada conserva índice y contenido.
Maki · calidad.py	perfilar(registros, fuente) -> dict: filas, campos, tipos observados, nulos, vacíos, duplicados y estadísticas. No cambia la entrada.
Maki · duplicados.py	separar_exactos(registros) -> dict con unicos y omitidos. detectar_conflictos(registros_normalizados) -> dict con candidatos y rechazados.
Avilix · integracion.py	normalizar(registros, fuente) -> dict con candidatos y rechazados. cruzar_catalogo(registros, catalogo) -> dict con candidatos y rechazados.
Leo · persistencia.py	clasificar_existentes(registros, conexion) -> dict con nuevos, ya_cargadas y rechazados. Comparar huella antes de imputar.
Maki · limpieza.py	imputar(registros, parametros=None) -> dict con candidatos, rechazados y parametros. aplicar_iqr(registros, parametros=None) -> dict con candidatos y parametros.
Leo · validacion.py	prevalidar(registros), validar_basico(registros) y validar_final(registros) -> dict con validos y rechazados. Prevalidar admite faltantes imputables, pero excluye defectos conocidos antes de estimar medianas.
Avilix · transformacion.py	escalar_segmentar(registros, parametros=None) -> dict con candidatos, rechazados y parametros.
Leo · persistencia.py	guardar_lote(resultado, conexion) -> dict con id_ejecucion, estado y conteos. Commit de todo o rollback de todo.
Avilix · pipeline.py	ejecutar_lote(ruta, fuente, ruta_catalogo, ruta_bd) -> dict con estado, id_ejecucion, conteos y rutas de evidencia. Importa funciones; no duplica su lógica.

Cada función acepta y devuelve listas de diccionarios; pandas puede usarse internamente. Las pruebas pueden usar datos sintéticos antes de recibir el módulo de otro compañero. No se importan funciones inexistentes para fingir una integración terminada.

Campos, tipos y orden del proceso

Grupo	Campos y reglas
Identidad	fuente: ATM o DIGITAL; id_transaccion e id_cuenta: str; id_ejecucion e id_lote: identificadores del proceso; origen_ingreso: ARCHIVO o MANUAL.
Evidencia	archivo_origen, posicion_origen (desde 1, documentando si incluye encabezado), original_json, huella_origen, transformaciones (lista de campo, antes, después y regla).
Valores comunes	fecha_hora: datetime con zona UTC-06:00; fecha: YYYY-MM-DD; tipo_operacion; estado_operacion; moneda_original; monto_original y monto_mxn: Decimal o None cuando proceda.
Campos específicos	ATM: id_cajero y comision_aplicada. Digital: beneficiary_clabe, notification_email, concept, platform. Conservar CLABE como texto; campos no aplicables se distinguen de faltantes.
Enriquecimiento	estado_cuenta, version_catalogo y atributos de cuenta usados. No usar limite_credito_mxn como saldo disponible.
Analítica	monto_analitico_mxn, es_outlier, monto_normalizado, segmento; parámetros y versión de reglas/proceso. Referencia estadística identificada para ingreso manual.
Error común	fuente, posición, original_json, id_transaccion disponible, errores: lista de {codigo, campo, mensaje}, fase, id_ejecucion y motivo_rechazo_cuarentena.
Serialización	Decimal se serializa como cadena decimal exacta y datetime como ISO con zona. En SQLite, dinero en centavos INTEGER; tasas/parámetros con precisión documentada. No usar float para persistir dinero.

Orden de ejecución que todos deben respetar

Lectura + trazabilidad → perfilado crudo → duplicados exactos → normalización y huella → conflictos internos y claves ya guardadas → cruce de catálogo → prevalidación → imputación → validación básica → IQR → Min-Max/segmentos → validación final → persistencia → métricas finales.

Solo registros nuevos y elegibles ajustan estadísticas. Duplicados, huérfanos y errores conocidos no contaminan los parámetros. Los valores no numéricos presentes se rechazan; no se convierten en nulos para imputarlos.

Identidad de una fila: fuente + posición en el lote. Cada fila procesada acaba en un solo destino: nueva limpia, cuarentena, duplicado interno o ya cargada. Los errores de una fila se acumulan, no se insertan como varias filas de cuarentena.

06 · MAKI · SÁBADO

Calidad y limpieza: lo que construyes

Tu trabajo: explicar qué está mal en las fuentes y corregir únicamente lo permitido. No haces la conexión SQLite ni las pantallas.

M1. Diagnóstico inicial · src/calidad.py

• Recibir registros originales del lector. Contar filas, nulos, vacíos, tipos por columna, duplicados exactos y categorías. Distinguir texto numérico de número real.

• Generar outputs/calidad_inicial.json con métricas por fuente. No sumar campos no aplicables como faltantes.

• Reproducir los controles iniciales: 50 cuentas; ATM 620 filas, 20 duplicados, 27 montos ausentes y 44 cajeros ausentes; digital 600 filas. Los errores pueden superponerse.

M2. Duplicados del mismo lote · src/duplicados.py

• Conservar primera fila exacta y devolver las adicionales en omitidos con posición y referencia a la conservada.

• Después de normalizar, detectar claves iguales con contenido distinto. Devolver las versiones conflictivas a cuarentena; no decidir cuál es verdadera.

• No quitar filas solo porque comparten cuenta, importe o fecha. Leo revisa los duplicados contra la base de datos.

M3. Faltantes e imputación · src/limpieza.py

• Usar los grupos de las reglas v1.4: ATM por operación/resultado; digital por moneda/estado; comisión por operación/cajero. Cinco observaciones elegibles como mínimo.

• Mediana solo de valores observados válidos. Consulta de saldo ausente se completa con 0; cajero con DESCONOCIDO; concepto con SIN_CONCEPTO. No inventar cuenta, CLABE ni identidad.

• Si no existe base estadística, devolver SIN_BASE_PARA_IMPUTAR. Guardar mediana, tamaño de grupo y cambios. No alterar original_json.

• Analizar la asociación de faltantes con operación, cajero y estado para formular hipótesis MCAR/MAR. Explicar límites; no afirmar causalidad ni inventar conclusiones.

> **Puedes empezar sin esperar al lector:** usa una lista pequeña de diccionarios según el contrato. Para validar tus resultados reales, espera la lectura y normalización de Avilix. Para ajustar IQR sobre población válida, espera la validación básica de Leo.

Tus entregables, pruebas y commits

M4. IQR y comparación final

En src/limpieza.py calcula Q1/Q3 con interpolación lineal e IQR 1.5, por fuente/operación, después de convertir a MXN. Excluye consultas y montos imputados del ajuste. Aplica límites a monto_analitico_mxn; preserva monto_mxn. Grupos menores de cinco o IQR cero se registran sin recorte.

En src/calidad.py agrega comparar_calidad(antes, despues, resumen) para generar outputs/calidad_final.json y outputs/comparacion_calidad.csv. Mostrar el efecto de imputaciones, rechazos y duplicados por separado; una tabla limpia no prueba por sí misma que la calidad mejoró.

Commit exacto	Archivos y prueba para poder subirlo
feat(calidad): agregar perfilado y duplicados del lote	src/calidad.py, src/duplicados.py, tests/test_calidad.py. Probar duplicado exacto, clave conflictiva, nulo, vacío y ausencia de mutación del original.
feat(limpieza): imputar faltantes con trazabilidad	src/limpieza.py, tests/test_limpieza.py. Probar consulta cero, mediana sesgada, cuatro vs. cinco observaciones, dato corrupto y grupos separados.
feat(limpieza): aplicar iqr y comparar calidad	Actualizar limpieza/calidad y sus pruebas. Comprobar límites, preservación del importe, IQR cero y comparación inicial/final.
docs(calidad): documentar resultados y justificacion	docs/reporte_maki.md. Incluir tablas reales, hipótesis MCAR/MAR, mediana vs. media, imputaciones y capping por grupo; sin cifras inventadas.

Qué entregas a los compañeros

A Avilix: funciones, esquema de parámetros y casos de prueba, indicando el commit integrado en main. A Leo: códigos de rechazo y estructura de evidencia. Tus funciones devuelven resultados; no guardan en SQLite ni cambian el catálogo.

Cómo compruebas que acabaste

```
python -m pytest tests/test_calidad.py tests/test_limpieza.py -q
```

El domingo compara las cifras del pipeline real, revisa tu sección del PDF final y ensaya por qué elegiste mediana, qué significa IQR y qué pasa cuando el grupo tiene pocos datos. Explica al menos un registro corregido y uno que no se pudo imputar.

Integración y transformación obligatorias

A1. Lectura · src/lectura.py

Implementa CSV UTF-8 con BOM opcional, XLSX en hoja transacciones_atm y JSON como lista. Valida encabezados, conserva filas y posiciones. Rechaza archivos ilegibles o con estructura global inválida; un elemento JSON incorrecto conserva evidencia para cuarentena. Detecta fórmulas de Excel sin ejecutarlas. El catálogo no se infiere de las transacciones.

A2. Homologación · src/integracion.py

• Mapea columnas ATM/digital al contrato. Normaliza identificadores y estados, conserva acentos de textos descriptivos y CLABEs como cadenas.

• Interpreta Excel y fechas ISO, Unix en segundos y UTC-06:00 según las reglas. Deriva YYYY-MM-DD sin borrar la hora original.

• Convierte USD a MXN con Decimal y tasa fija 19.50, redondeo HALF_UP a centavos; ATM ya está en MXN.

• Construye la representación normalizada de origen antes de imputar y su huella SHA-256. No incluyas datos de catálogo, lote ni variables derivadas.

• Cruza con el catálogo sin multiplicar filas. Devuelve CUENTA_NO_REGISTRADA para huérfanos y observa cuentas bloqueadas/suspendidas sin inventar su estado histórico.

A3. Transformación · src/transformacion.py

Calcula Min-Max y cuantiles 1/3 y 2/3 sobre montos analíticos válidos, por fuente/operación. Trata rango constante, cortes coincidentes y consultas de saldo. Guarda mínimos, máximos y cortes por ejecución; no recalcula datos históricos. Un grupo vacío no genera valores ficticios.

A4. Coordinación · src/pipeline.py y src/cli.py

Conecta etapas en el orden de la página 6, acumula rechazos y omitidos sin duplicarlos y llama a guardar_lote de Leo. La consola recibe ruta de archivo, fuente, catálogo y BD. La implementación principal es Python aunque los extras usen HTML/CSS/JavaScript.

```
python -m src.cli cargar --fuente ATM --archivo datasets/transacciones_atm.xlsx --catalogo datasets/catalogo_cuentas.csv --bd outputs/pipeline.sqlite
```

Ese comando es el contrato que debes implementar. DIGITAL usa el JSON y la misma BD. Agrega subcomandos resumen, actualizar-catalogo y reprocesar con ayuda --help y ejemplos exactos en README.

No esperes para todo: lector, normalización, cruce y pruebas sintéticas pueden hacerse desde temprano. La conexión real espera las funciones M1-M4 y L1-L3, no las pantallas.

09 · AVILIX · SÁBADO Y DOMINGO

Tus commits y la integración

Commit exacto	Archivos y condición de salida
feat(ingesta): leer fuentes bancarias con trazabilidad	src/lectura.py y tests/test_integracion.py. Leer las tres fuentes y comprobar columnas, posiciones, formatos y archivo inválido.
feat(integracion): normalizar fechas monedas y cuentas	src/integracion.py y tests/test_integracion.py. Probar 100 USD = 1950 MXN, CLABE con cero inicial, Unix y cuenta huérfana.
feat(transformacion): escalar y segmentar importes	src/transformacion.py y tests/test_transformacion.py. Probar rango constante, grupos vacíos, límites y cortes coincidentes.
feat(pipeline): conectar procesamiento y consola	src/pipeline.py, src/cli.py, tests/test_pipeline.py. Espera los módulos base de Maki y Leo; ejecutar sobre SQLite temporal y reconciliar destinos.
docs: completar diagrama y guia de ejecucion	README.md, docs/diagrama_pipeline.mmd, docs/diagrama_pipeline.png y docs/reporte_avilix.md. Los comandos publicados ya deben funcionar.
docs: preparar reporte y entrega final	scripts/generar_reporte.py, delivery/REPORTE_PROYECTO.pdf, delivery/pipeline_banco.sqlite, delivery/RESULTADOS_PRUEBAS.txt, delivery/MANIFIESTO_ENTREGA.md. Solo después de las verificaciones del domingo.

Responsabilidades de coordinación

• Revisa y fusiona las ramas de Maki y Leo sin atribuirte sus commits. Conserva sus autores y pide correcciones en el archivo de su propiedad.

• Integra temprano: el primer recorrido completo puede usar 10 registros sintéticos. No esperes a que todas las funciones estén perfectas.

• Actualiza REGLAS_DE_NEGOCIO.md únicamente cuando una decisión cambie de verdad y los tres conozcan su efecto. Añade los nombres y evidencia de firma real, sin firmar por tus compañeros.

• Reúne los tres textos del reporte; comprueba las cifras con Maki y las tablas de BD con Leo.

Pruebas propias: python -m pytest tests/test_integracion.py tests/test_transformacion.py tests/test_pipeline.py -q. Los comandos y pruebas solo se anuncian como exitosos después de ejecutarse.

10 · LEO · SÁBADO

Validación, cuarentena y SQLite

L1. Modelos · src/validacion.py

Usa Pydantic v2, BaseModel, @field_validator y validadores de modelo para relaciones entre campos. La conversión explícita ya la hizo Avilix; valida estrictamente el resultado. Crea modelos ATM, digital y final común.

• Consulta de saldo exige 0; retiro, depósito, servicios y transferencia exigen monto positivo. Comisiones no negativas; números finitos.

• CLABE digital: exactamente 18 dígitos ASCII, conservados como texto. No declarar verificación bancaria ni validar dígito de control no exigido.

• Correo opcional: vacío a nulo; presente con EmailStr. Estados y operaciones según las reglas. Un estado bancario fallido puede ser un registro válido.

• Valida identidad, fecha, moneda, cuenta existente, campos de cada fuente y analítica final. Campo no aplicable no es error.

L2. Errores · src/cuarentena.py

Captura ValidationError por fila y devuelve todos sus motivos detectados. Consolida por identidad de fila con errores de conversión, cuenta o imputación. Conserva original, campo, código, fase, fecha e identificadores. No uses except general para esconder errores de programación.

L3. Persistencia · src/schema.sql y src/persistencia.py

Crea transacciones_limpias y cuarentena_errores, más ejecuciones, catalogo_cuentas, versiones_catalogo, parametros_estadisticos y bitacora_omitidos. Incluye restricciones, claves foráneas activadas y UNIQUE(fuente, id_transaccion) global. Guarda dinero en centavos; las consultas reconstruyen la presentación decimal.

Guarda limpias, rechazos, omitidos, parámetros y cierre de ejecución en una sola transacción. Un error provoca rollback completo. Registra FALLIDA en una transacción posterior, sin afirmar que filas revertidas se guardaron. Usa consultas parametrizadas.

> **Puedes trabajar desde el contrato:** modelos y SQLite se prueban con datos sintéticos. La prueba de carga real espera candidatos de Avilix/Maki. No necesitas esperar al dashboard.

Tus commits y las cargas posteriores

Commit exacto	Archivos y condición de salida
feat(validacion): agregar modelos y cuarentena	src/validacion.py, src/cuarentena.py, tests/test_validacion.py. Probar montos cero/negativos, correo, CLABE, categorías y múltiples motivos por fila.
feat(bd): persistir lotes con commit y rollback	src/schema.sql, src/persistencia.py, tests/test_persistencia.py. Comprobar inserción completa, fallo deliberado a mitad de carga y cero filas parciales.
feat(cargas): controlar duplicados y catalogos	src/catalogo.py, actualizar persistencia y pruebas. Huellas de archivo/operación, versión de catálogo y unicidad entre lotes; mismo archivo renombrado no se recarga.
feat(cuarentena): reprocesar sin perder historial	Actualizar cuarentena/persistencia; src/consultas.py. Enlazar intentos y resolver solo tras inserción o repetición válida confirmada; no borrar el rechazo original.
test: comprobar resiliencia y documentar resultados	tests/test_resiliencia.py, docs/reporte_leo.md. Estrés, invalidaciones de archivo, lotes repetidos, conflictos, catálogo nuevo y pruebas de fallo.

L4. Catálogo y duplicados globales

src/catalogo.py valida la versión completa antes de activarla: no borrar cuentas anteriores, no aceptar claves repetidas, estados desconocidos o límites negativos. Conserva versiones. src/persistencia.py clasifica claves existentes: misma huella = YA_CARGADA; distinta = cuarentena, sin modificar la fila guardada.

L5. Consultas para el equipo

src/consultas.py expone obtener_resumen(filtros), listar_transacciones(filtros, pagina, tamano), obtener_detalle(id), listar_cuarentena(filtros) y listar_ejecuciones(filtros). Devuelve dict/list, totales globales y paginación; no HTML. Avilix los usa en la interfaz. Los filtros se parametrizan y se prueban.

```
python -m pytest tests/test_validacion.py tests/test_persistencia.py tests/test_resiliencia.py -q
```

El domingo entrega a Avilix una BD generada por el pipeline, resultados reales de pruebas y explicación de las tablas. No generes una BD manualmente para simular que el pipeline funciona.

Cuándo esperar y qué hacer mientras

Necesita	Espera este resultado	Mientras tanto puede hacer
Todos · inicio sábado	Avilix: contrato, estructura y fuentes accesibles en main.	Leer reglas y este plan; configurar Git y entorno local.
Maki · datos reales	Avilix: leer_fuente y normalizar funcionando.	Perfilado, imputación y pruebas con dicts sintéticos.
Avilix · Min-Max real	Maki: monto_analitico_mxn; Leo: lista de válidos básicos.	Probar fórmula y cortes con valores conocidos.
Leo · guardar lote real	Avilix: resultado común del pipeline; Maki: errores y parámetros.	Schema, restricciones, rollback y validadores con fixtures propios.
Avilix · primer flujo	Maki: M1-M4; Leo: L1-L3 integrados.	Lectores, normalización, CLI y prueba de integración preparada.
Todos · nuevos lotes	Leo: control global de duplicados y catálogo versionado.	Revisar ejemplos superpuestos y preparar pruebas.
Avilix · dashboard	Leo: consultas; pipeline completo con BD confirmada.	Diseñar HTML/CSS y navegación local; etiquetar maquetas como tales.
Avilix · ingreso manual	Leo: validación/persistencia; Maki y Avilix: parámetros guardados.	Construir el cajero y la vista digital sin afirmar que ya guardan.
Reporte final	Maki: calidad; Leo: pruebas/BD; Avilix: integración/diagrama.	Escribir método y estructura; resultados esperan la ejecución real.

Una dependencia está lista cuando: su commit está integrado en main, la función respeta el contrato y pasó su prueba. Un mensaje de “ya casi” o un archivo enviado por chat no sustituye la integración.

Si una función falta, la IA puede usar un fixture de prueba claramente identificado. No se sube a main una función que siempre devuelve éxito ni una pantalla con números ficticios presentados como resultados.

Aviso de entrega entre compañeros: “Módulo X listo; commit abc123; función Y; prueba Z pasó; ya puedes ejecutar W”. Si cambia una firma o campo, detener la integración afectada y actualizar el contrato con Avilix.

Viernes, sábado y domingo

Momento / salida	Avilix	Maki	Leo
Viernes 23 Preparación	Sube plan, contrato, estructura y README. Invita colaboradores.	Lee plan y reglas; acepta invitación. Sin commits.	Lee plan y reglas; acepta invitación. Sin commits.
Sábado 24 Primer bloque	Lector y normalización A1/A2; pruebas de formato.	Perfilado y duplicados M1/M2 con fixtures.	Modelos, cuarentena y esquema L1/L2.
Sábado Segundo bloque	Cruce, Min-Max y segmentos A3. Revisa PR tempranos.	Imputación e IQR M3/M4; prueba casos límite.	Persistencia L3 y pruebas de rollback.
Sábado Cierre obligatorio	Integra A4, ejecuta lote real y revisa errores compartidos.	Revisa métricas y transformaciones reales.	Comprueba ambas tablas, rechazos y no-crash.
Domingo 25 Primer bloque	Conecta nuevas cargas y congela el núcleo cuando pase pruebas.	Completa comparación, hipótesis y reporte de calidad.	L4/L5: lotes repetidos, catálogo, reproceso y estrés.
Domingo Segundo bloque	Extras visuales sobre servicios probados; después arma reporte.	Revisa dashboard contra cifras reales; redacta su defensa.	Verifica consultas, comprobantes y BD de entrega.
Domingo Cierre	Genera PDF y manifiesto, merge final, prueba clon limpio.	Valida calidad, firma real y ensaya explicación.	Valida BD, firma real y ensaya errores/rollback.

Puntos de control que no se saltan

• G0 viernes: los tres conocen archivos y firmas. G1 sábado: cada módulo tiene al menos una prueba pasada.

• G2 sábado: una ejecución completa produce ambas tablas y reconcilia las entradas. Si falla, el domingo empieza corrigiendo el núcleo.

• G3 domingo: cargas repetidas y datos corruptos no generan duplicados ni inserciones parciales.

• G4 cierre: los tres verifican la entrega desde una copia limpia y pueden explicar su parte.

No se fijan horas porque no se indicó la disponibilidad del equipo. Los bloques se ejecutan en este orden y las dependencias determinan el avance.

Cómo subir sin pisar el trabajo ajeno

Commit: una versión local de cambios. **Push:** subirla a GitHub. **Rama:** espacio de trabajo propio. **Pull request (PR):** solicitud para revisar e integrar una rama en main.

Una sola vez, en cada computadora

```
git clone https://github.com/Av1I1X/Pipeline_Banco.git
cd Pipeline_Banco
git config user.name "TU NOMBRE"
git config user.email "TU CORREO VERIFICADO EN GITHUB"
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Sustituir nombre/correo por los propios; se puede usar el correo noreply de GitHub. Nunca usar la identidad de otro compañero. Los comandos posteriores con python se ejecutan con el entorno activado o con .venv\Scripts\python.exe.

Al comenzar cada bloque, desde una copia sin cambios pendientes

```
git switch main
git pull --ff-only origin main
git switch -c maki/calidad
```

Maki usa maki/calidad; Leo, leo/validacion-bd; Avilix, avilix/integracion. Después del merge, crear una rama nueva para el siguiente bloque. Los extras de Avilix van en avilix/interfaz.

Después de implementar y probar

```
git status
git add src/calidad.py tests/test_calidad.py
git diff --cached
git commit -m "feat(calidad): agregar perfilado y duplicados del lote"
git push -u origin maki/calidad
```

El git add es un ejemplo: cada persona agrega sus archivos reales y el mensaje correspondiente de su página. No copiarlo si esos archivos no cambiaron. Revisar que no se agregaron secretos, entornos, salidas temporales o trabajo de otro.

En GitHub, abrir Compare & pull request hacia main. Escribir: qué cambió, archivos, prueba ejecutada y qué desbloquea. Avilix revisa y fusiona conservando los commits originales. Si hay conflictos, resolverlos con el dueño del archivo; nunca usar push --force ni borrar trabajo para “arreglar” Git.

Pedir ayuda sin perder el control

La IA ayuda a escribir y revisar, pero cada integrante debe leer el código, ejecutar pruebas y explicar sus decisiones. Pedir tareas pequeñas, por commit, en vez de “haz todo el proyecto”.

Texto base que cada uno pega al iniciar su sesión

> Trabajo en Pipeline_Banco. Lee REGLAS_DE_NEGOCIO.md v1.4, docs/CONTRATO_DATOS.md y docs/PLAN_DE_TRABAJO.pdf. Mi nombre es [nombre] y mi rol es [rol]. Implementa únicamente el bloque [código del bloque] y sus pruebas. Respeta firmas, rutas y propiedad de archivos. No cambies reglas ni módulos ajenos sin explicar la necesidad. No inventes resultados. Ejecuta las pruebas y muestra su salida. Antes de subir, muéstrame el diff y los archivos incluidos. Usa mi identidad Git y no añadas coautores automáticos.

Primer pedido específico por integrante

Persona	Pedido
Maki	Implementa M1/M2 en src/calidad.py y src/duplicados.py. Usa datos sintéticos para pruebas y conserva originales. Entrega las salidas acordadas; no escribas a SQLite.
Avilix	Implementa A1/A2 en lectura.py e integracion.py. Lee las fuentes sin alterar originales; respeta Decimal, fechas y CLABE texto. No implementes todavía el dashboard.
Leo	Implementa L1/L2 con Pydantic v2 y errores por fila. Prueba CLABE, correo, monto y varios motivos. No implementes otro lector ni limpieza estadística.

Al cerrar cada bloque, exigir estas respuestas

- ¿Qué archivos cambiaste y qué función pública entregas?
- ¿Qué pruebas ejecutaste y qué salida real obtuviste? Si no se ejecutó, indícalo.
- ¿Qué regla del .md respalda cada decisión y qué dependencia falta?
- Explícame un caso válido, uno rechazado y cómo se conserva su evidencia.
- ¿Qué commit debo subir y qué puede empezar ahora mi compañero?

Para un error, copiar el comando y el traceback completo, eliminando credenciales si existieran. No pedir a la IA que comente las validaciones o devuelva éxito fijo para hacer pasar las pruebas.

16 · VERIFICACIÓN CONJUNTA

Pruebas que deben quedar demostradas

Caso	Resultado esperado	Dueño
Fuentes iniciales sobre BD vacía	1,220 transacciones; 20 duplicados exactos ATM; 1,200 candidatos antes de otros rechazos. Catálogo: 50 cuentas.	Avilix + Maki
Montos y faltantes	Consulta cero válida; negativos a cuarentena; medianas con mínimo cinco; texto corrupto no imputado.	Maki + Leo
CLABE y correo	46 CLABEs cortas identificables en el JSON inicial. Correo: 8 nulos, 8 vacíos, 30 errores por revisión básica; explicar diferencias de EmailStr.	Leo
Huérfanos	ATM 32 y digital 29 en las entradas crudas; los conteos finales pueden cambiar al deduplicar.	Avilix
IQR y analítica	Monto original conservado; límites IQR correctos; escala [0,1]; grupos vacíos/constantes controlados.	Maki + Avilix
Lote parcialmente corrupto	Filas buenas continúan; cada mala guarda evidencia y motivos; no hay caída por datos previstos.	Leo
Falla de persistencia	Rollback completo; ninguna inserción parcial. Registro de ejecución fallida.	Leo
Carga repetida y superpuesta	Archivo igual renombrado se omite. Misma clave/huella no duplica; clave igual con cambios a cuarentena.	Leo + Avilix
Catálogo/reproceso	Nueva cuenta permite reintentar; catálogo inválido no reemplaza el vigente; historial de rechazo se conserva.	Leo
Interfaces extras	Reintento manual no duplica; referencia estadística identificada; cancelación previa no guarda; gráficas coinciden con SQL.	Avilix + Leo

```
python -m pytest -q
```

Pruebas usan SQLite temporal y fixtures sintéticos propios de cada test. No escriben sobre la BD de entrega ni modifican los originales. Guardar el comando, fecha, resultado y cantidad de pruebas en delivery/RESULTADOS_PRUEBAS.txt; no copiar una salida de ejemplo.

Reconciliación por ejecución confirmada: filas leídas = nuevas limpias + cuarentena + duplicados internos + ya cargadas. Los motivos de error pueden ser varios por fila; no se suman como si fueran filas distintas.

Dashboard y dos experiencias visuales

Se construyen después de G2. Arquitectura acordada: FastAPI como puente Python y HTML/CSS/JavaScript para las vistas. Esto permite un cajero visual real en pantalla sin duplicar la lógica ETL. La interfaz no ejecuta operaciones bancarias reales.

Archivo de Avilix	Contenido obligatorio del extra
src/api.py	Rutas para cargar archivo, ingresar transacción, consultar resumen/detalle e historial y reprocesar. Llama a pipeline/consultas; maneja solicitud única y estados reales.
web/index.html web/css/dashboard.css web/js/dashboard.js	Indicadores, filtros, tabla paginada de limpias, panel original/transformado, cuarentena, historial y dos gráficas: resultados por lote y motivos de rechazo.
web/atm.html web/css/atm.css web/js/atm.js	Pantalla tipo cajero, botones laterales, teclado numérico y centavos; operaciones, revisar/corregir/cancelar/confirmar y comprobante. No mostrar saldo inventado.
web/digital.html web/css/digital.css web/js/digital.js	Estilo app bancaria: cuenta origen, CLABE, importe, moneda, concepto, correo, revisión, equivalente MXN y comprobante.
web/css/base.css web/js/api.js	Estilos compartidos, acceso por teclado, comunicación API y estados de carga/error. Ningún dato ficticio presentado como definitivo.
tests/test_api.py docs/PRUEBAS_INTERFAZ.md	Pruebas de servicios y registro de revisión visual: móvil/escritorio, navegación, cancelación, duplicados, errores y consultas sin datos.

Commits de extras, en este orden

```
feat(api): exponer servicios del pipeline
feat(dashboard): mostrar control y detalle de cargas
feat(atm): agregar simulador visual de cajero
feat(digital): agregar experiencia de transferencia
test(ui): verificar flujos visuales y resultados
```

La API recibe rutas de operación y datos validados, no SQL libre. No crear archivos usando rutas arbitrarias enviadas por el navegador. El acceso de archivos se limita al flujo de carga; no publicar un servidor con credenciales o datos personales reales.

Cómo guardar desde el cajero o la app

Contrato entre pantalla y backend

Avilix documenta en docs/CONTRATO_DATOS.md las rutas: POST /api/cargas, POST /api/transacciones, GET /api/resumen, GET /api/transacciones, GET /api/transacciones/{id}, GET /api/cuarentena, GET /api/ejecuciones y POST /api/cuarentena/{id}/reprocesar. Leo entrega las consultas; Avilix conecta HTTP sin reescribir SQLite.

Una solicitud manual incluye solicitud_id, fuente, datos_origen y referencia_ejecucion. Avilix implementa ejecutar_manual(solicitud, ruta_bd) en src/pipeline.py; la API la llama y reutiliza los módulos existentes. Devuelve id_ejecucion, resultado, motivos y detalle. El backend valida todo aunque el navegador ya lo haya revisado.

Referencia estadística: dependencia obligatoria del ingreso manual

• Maki guarda medianas, tamaños de grupo y límites IQR; Avilix guarda mínimos, máximos y cortes; Leo persiste los parámetros vinculados a ejecución y versión.

• Avilix permite seleccionar una ejecución de archivos completada y compatible. Una sola fila no se usa para ajustar sus propios parámetros.

• Sin referencia o grupo requerido: REFERENCIA_ESTADISTICA_INSUFICIENTE. Consulta de saldo: excepción determinista, monto y escala 0, segmento BAJO.

• Respetar rango constante y recorte de Min-Max según la sección 16.2 de las reglas. No cambiar los parámetros ni registros de la carga de referencia.

Qué se muestra al confirmar

INSERTADA, EN_CUARENTENA o YA_CARGADA, únicamente después del commit. FALLIDA corresponde a error técnico. El estado bancario simulado se muestra por separado. El doble clic se controla tanto en pantalla como mediante solicitud_id en persistencia.

Detalles visuales que no se omiten

• ATM: cuenta y cajero, tipo de operación, teclado, comisión/datos de simulación, revisión, corrección y comprobante. Cancelar antes de enviar no crea una fila.

• Digital: cuenta, CLABE texto, correo opcional, importe/divisa, concepto, plataforma, estado y fecha; confirmación “Registrar transferencia simulada”.

• Dashboard: totales completos aunque la tabla tenga paginación; rechazos abiertos separados de históricos; dos filtros de fecha claramente identificados.

• Acciones útiles con teclado, botones funcionales en móvil y conservación de lo escrito ante errores. No pedir PIN, tarjeta ni credenciales bancarias reales.

Cómo armar la entrega académica

Archivo	Qué debe llevar / responsable
docs/reporte_maki.md	Diagnóstico cuantitativo, hipótesis MCAR/MAR, tablas de mediana/media, IQR, cambios antes/después y límites del análisis. Maki escribe y comprueba.
docs/reporte_avilix.md	Objetivo, arquitectura, integración, formatos, fechas/divisas, Min-Max/segmentos, decisiones y extras diferenciados. Avilix escribe.
docs/reporte_leo.md	Pydantic, motivos, esquema de tablas, transacciones, pruebas de corrupción/rollback/repetición, resultados y limitaciones. Leo escribe.
delivery/REPORTE_PROYECTO.pdf	Avilix ensambla: portada con nombres, objetivo, diagrama, reglas justificadas, calidad antes/después, métodos, resultados, BD, pruebas, conclusiones y referencias. Los tres revisan su sección.
docs/FIRMAS_REGLAS.pdf	Los tres firman realmente la justificación o una hoja que identifica REGLAS_DE_NEGOCIO.md, versión y commit. Avilix enlaza esa evidencia desde el .md. No fabricar firmas ni confundir autoría Git con firma sin aceptación docente.
delivery/pipeline_banco.sqlite	Leo verifica; Avilix incluye la copia final cerrada de SQLite, sin archivos WAL pendientes. Debe contener las dos tablas exigidas y datos de la ejecución demostrada.
delivery/MANIFIESTO_ENTREGA.md	Commit de código/reglas usado, archivos fuente y hashes, versiones, comando real, ejecución final, conteos, pruebas y lista de archivos entregados. Separar registros sintéticos de datos oficiales.

Defensa oral de 5 a 7 minutos

Avilix, 2 minutos: objetivo, diagrama, formatos y demostración de ejecución. **Maki, 2 minutos:** un faltante, un outlier y comparación antes/después. **Leo, 2 minutos:** registro rechazado, tablas y prueba de rollback/no-crash. Si queda tiempo, Avilix muestra el extra visual; no consumir la defensa solo en el diseño.

Todos deben explicar el flujo completo y su propio código. Preparar respuestas: por qué mediana, por qué consulta cero es válida, qué sucede con una cuenta huérfana, cómo se evita duplicar y qué conserva el rollback.

20 · CIERRE

Lista final para no dejar pendientes

Comprobar antes de entregar	Responsable
Roles publicados y aportaciones identificables de los tres en Git.	Avilix; verifica cada autor
Fuentes originales conservadas; ruta y procedencia documentadas.	Avilix
Diagrama completo con rama de cuarentena y errores de archivo/BD.	Avilix + revisión de Leo
Reglas con justificación y firmas reales vinculadas a su versión.	Los tres
Perfilado inicial, MCAR/MAR justificado y evidencia antes/después.	Maki
Medianas, IQR 1.5, monedas, fechas, huérfanos, Min-Max y segmentos.	Maki + Avilix
Pydantic con decoradores, motivos por fila y tablas obligatorias.	Leo
Carga completa, rollback, estrés, no-crash y reconciliación verificados.	Leo + Avilix
Nuevos archivos, duplicados globales, catálogo y reproceso probados.	Leo + Avilix
Dashboard y ambas interfaces conectados o estado pendiente declarado con precisión; nunca resultados simulados como reales.	Avilix
PDF final legible, SQLite utilizable y README ejecutable desde clon limpio.	Los tres
Defensa ensayada y cada integrante comprende su parte.	Los tres

Prueba final desde una copia limpia

Una persona clona el repositorio en otra carpeta, instala dependencias, coloca las fuentes, ejecuta las instrucciones del README y corre pytest. Genera una BD nueva de comprobación y compara conteos. No sobrescribe la BD final. Si un paso necesita conocimiento que no está escrito, corregir el README.

Fuentes y relación con la rúbrica

Base: GUIA_LINEAMIENTOS_PROYECTO_ALUMNOS.pdf, páginas 1-3; REGLAS_DE_NEGOCIO.md v1.4; análisis de catalogo_cuentas.csv, transacciones_atm.xlsx y transferencias_digitales.json. Rúbrica: mockup/reglas 10; calidad 20; integración 20; Pydantic/cuarentena 20; BD 15; resiliencia/defensa 15. Total: 100 puntos; el proyecto representa 60% del portafolio de la unidad. Los extras no tienen puntuación adicional garantizada.

Entrega terminada significa: código que corre, pruebas reales, datos trazables, base de datos coherente, documentación completa y tres personas capaces de explicarlo. Un commit con archivos no equivale por sí solo a una función terminada.